Week 3: Functions and Arrays

1. Creating and calling functions in PHP
   1.1. Definition

In PHP, functions are blocks of reusable code that perform specific tasks. Functions help organize code, improve code reusability, and make it easier to maintain and debug your PHP scripts. You can create your own functions or use built-in functions provided by PHP. Here's an overview of defining and using functions in PHP.

To define a custom function in PHP, you use the function keyword followed by the function name, a pair of parentheses, and a set of curly braces containing the function's code block.

Syntax:

```php
function functionName(parameters) {
    // Function code here
}
```

Example:

```php
function greet($name) {
    echo "Hello, " . $name . "!";
}
```

1.2. Calling a Function

Once you've defined a function, you can call it by using its name followed by a pair of parentheses. If the function has parameters, you need to pass the appropriate values inside the parentheses.

Example:

```php
greet("John"); // Outputs: Hello, John!
```

1.3. Return Values

Functions can return values using the `return` keyword. If a function doesn't have a `return` statement, it implicitly returns `null`.
Example:

```php
function add($a, $b) {
    return $a + $b;
}

$result = add(5, 3);
echo $result; // Outputs: 8
```

1.4. Default Parameters:

You can set default values for function parameters. If a parameter is not passed when calling the function, the default value will be used.
Example:

```php
function greetDefault($name = "Guest") {
    echo "Hello, " . $name . "!";
}

greetDefault(); // Outputs: Hello, Guest!
greetDefault("Alice"); // Outputs: Hello, Alice!
```

1.5. Variable Scope:

PHP has different variable scopes: global and local. Variables declared outside of a function are global, while those declared inside a function are local and accessible only within that function unless explicitly declared as `global`.

1.6. Anonymous Functions (Closures):

PHP also supports anonymous functions, also known as closures. These functions can be assigned to variables and used as callback functions.
Example:

```php
$addition = function ($a, $b) {
    return $a + $b;
};

echo $addition(2, 3); // Outputs: 5
```

1.7. Function Arguments:

PHP functions can accept a variable number of arguments using the **`func_get_args()`** or the **`func_get_arg()`** functions. Starting from PHP 5.6, you can also define variadic functions using the `...` operator.
Example:

```php
function sum(...$numbers) {
    return array_sum($numbers);
}

echo sum(1, 2, 3, 4); // Outputs: 10
```

2. Understanding arrays and their types in PHP

Arrays are essential data structures in PHP, allowing you to store multiple values under a single variable name. Here's a brief overview of PHP functions related to arrays:

2.1. Creating Arrays

You can create arrays in several ways:

2.1.1. Using square brackets (indexed array):

```php
$numbers = [1, 2, 3, 4, 5];
```

2.1.2. Using the array() function (indexed array):

```php
$fruits = array('apple', 'banana', 'orange');
```

2.1.3. Associative arrays (key-value pairs):

```php
$person = array('name' => 'John', 'age' => 30, 'occupation' => 'Engineer');
```

2.2. Accessing Array Elements:

You can access array elements using their index (for indexed arrays) or their keys (for associative arrays):

```php
echo $numbers[0]; // Outputs: 1
echo $fruits[2]; // Outputs: orange
echo $person['name']; // Outputs: John
```

2.3. Array Functions:

PHP provides numerous functions for manipulating arrays. Some common array functions include:

- count(): Returns the number of elements in an array.
- array_push(): Adds one or more elements to the end of an array.
- array_pop(): Removes and returns the last element of an array.
- array_shift(): Removes and returns the first element of an array.
- array_unshift(): Adds one or more elements to the beginning of an array.
- array_merge(): Merges one or more arrays into a new array.
- array_keys(): Returns all the keys of an array.
- array_values(): Returns all the values of an array.

Here's an example of using some of these functions:

```php
$colors = ['red', 'green'];
echo count($colors); // Outputs: 2

array_push($colors, 'blue');
print_r($colors); // Outputs: Array([0] => red [1] => green [2] => blue)

array_pop($colors);
print_r($colors); // Outputs: Array([0] => red [1] => green)
```

2.4. Looping through Arrays:

You can use loops like **foreach** to iterate through the elements of an array:

```php
$fruits = array('apple', 'banana', 'orange');
foreach ($fruits as $fruit) {
    echo $fruit . " ";
}
// Outputs: apple banana orange
```

2.5. Multidimensional Arrays:

3. Array manipulation and iteration

PHP also supports multidimensional arrays, which are arrays of arrays:

```php
$matrix = array(
    array(1, 2, 3),
    array(4, 5, 6),
    array(7, 8, 9),
);

echo $matrix[1][2]; // Outputs: 6
```